

# AI-based Village Pond Planning System

CSD Assignment 1 – Final Technical Report

AMANDEEIP KAMMARI, [TODO: Institute / Department] (Roll No. 12341060), India

Village ponds are among the cheapest ways to harvest rainwater in rural India, but deciding where to dig one, and how big to make it, normally needs a field survey and a hydrologist. We present a web application that does this preliminary planning from free public data. Given a village name or a map click, the server builds a 20 m elevation model from global terrain tiles (or from an uploaded KML contour map), routes surface flow with Priority-Flood depression filling and the D8 algorithm, and identifies land available for excavation by combining an OpenCV classifier on satellite imagery with OpenStreetMap exclusions and optional government parcels. Candidate sites are scored on flow accumulation, slope, concavity and land suitability, and a ranking algorithm returns up to five sites whose catchments are hydrologically independent. Twenty years of daily CHIRPS rainfall drive a daily SCS Curve Number runoff model, and a pond is sized to half of the 75 %-dependable annual runoff. The system is a FastAPI monolith with an SQLite store, multi-level caching and fallback chains for every external service, behind a Leaflet front end. Thirty-five tests pass; CHIRPS rainfall is within 8.9 % of IMD station normals on average, and the land classifier is 92 % accurate ( $\kappa = 0.85$ ) on 100 blind-labelled cells. For Hiware Bazar (Maharashtra) the system recommends five ponds storing 121 155 m<sup>3</sup> in total. A first analysis takes about 44 s, dominated by the rainfall download; a cached analysis takes 1.3 s, and eight concurrent cached analyses complete in 1.3 s each.

Additional Key Words and Phrases: Village Pond Planning, Geospatial Analysis, Rainwater Harvesting, Catchment Estimation, Web GIS, REST APIs

## 1 INTRODUCTION

Most of India's net sown area is rain-fed, and most of its rain falls in the four months of the south-west monsoon. Small surface storages — farm ponds, village tanks and the *Amrit Sarovars* built under national programmes [1] — hold this water back for irrigation, livestock and groundwater recharge. A pond is only useful, however, if three early decisions are right: it must sit on a drainage line that actually receives runoff and on land the village is allowed to dig; its catchment must deliver enough water even in a dry year; and its volume must match that inflow, since a pond much larger than its dependable inflow rarely fills and one much smaller wastes the catchment.

The goal of this assignment is a web application that analyses terrain and rainfall to recommend suitable pond locations and to estimate catchment area, rainfall, runoff, pond depth and storage capacity, all shown on an interactive map. Section 2 states the requirements, Section 3 the architecture, Section 4 the algorithms and Section 5 the implementation. Section 6 maps the design to Computer System Design themes, Section 7 presents results and measured performance, and Section 8 discusses limitations.

### 1.1 Motivation

Choosing a pond site by hand is hard for four reasons. First, terrain is subtle: on the gently sloping Deccan plateau a drainage line can be invisible on the ground, and a few metres of relief decide where water collects. Second, rainfall data is scattered: IMD station records are sparse, gridded products differ widely, and a village administrator rarely knows the dependable (dry-year) rainfall that should govern design. Third, land ownership records are not openly available, so finding land that is both free and hydrologically useful needs local knowledge. Fourth, sizing requires a runoff model, which is beyond most panchayat staff. An automated tool that turns a village name into a short, quantified list of candidate sites lets an engineer spend field time verifying a handful of good options instead of searching a whole village.

## 1.2 Scope of the Project

The system **does**: find a village by name or map click; build a terrain model from an elevation service or an uploaded KML/KMZ contour map; display satellite imagery, contours, drainage and land cover; identify land available for excavation; recommend up to five ranked pond sites with independent catchments (or evaluate a site chosen by the user); delineate each catchment; retrieve 5–40 years of daily rainfall; estimate runoff; recommend pond dimensions, storage, earthwork and water losses; and save, list and export analyses.

The system **does not**: perform structural or geotechnical design of embankments, inlets or spillways; determine legal land ownership (it can only use parcels supplied by the administrator); model groundwater, soil infiltration measurements or water quality; estimate costs; or replace a field survey. It is a pre-feasibility tool.

## 2 PROBLEM STATEMENT AND REQUIREMENTS

Given a location in rural India, the system must recommend where a rainwater-harvesting pond should be excavated and how large it should be, and justify the recommendation with terrain, land, rainfall and runoff evidence shown on a map. Table 1 maps each functional requirement of the brief to the module that implements it.

Table 1. Functional requirements and where they are implemented

| Requirement                         | Implemented in            | How                                                                                    |
|-------------------------------------|---------------------------|----------------------------------------------------------------------------------------|
| Satellite imagery display           | geocode.py, static/app.js | Nominatim village search; Esri World Imagery basemap with labels                       |
| Contour map visualization           | layers.py, kml_parser.py  | Contours traced from the DEM with OpenCV, or the uploaded KML contours; relief overlay |
| Available-land identification       | landcover.py              | OpenCV image classification + OpenStreetMap exclusions + user parcels + slope limit    |
| Catchment area estimation           | hydrology.py              | Priority-Flood, D8, flow accumulation, reverse traversal, boundary tracing             |
| Historical rainfall query           | rainfall.py               | CHIRPS (ClimateSERV), fallback Open-Meteo ERA5, then NASA POWER                        |
| Runoff volume estimation            | runoff.py                 | Daily SCS Curve Number with antecedent-moisture adjustment; rational-method check      |
| Pond depth / storage recommendation | design.py, pond.py        | Site scoring and ranking; excavated frustum sized to dependable runoff                 |
| Combined overlay / results view     | static/, pipeline.py      | Leaflet layers, Chart.js charts, candidate table, JSON/GeoJSON export                  |

### 2.1 Non-Functional Requirements

**Performance** A first analysis of a new village should finish within about one minute, and a repeat analysis within 5 s. Lightweight endpoints (health, cached rainfall, geocoding) should answer in under 50 ms.

**Scalability** One instance should serve a small team of planners — up to about ten concurrent analyses — without degrading results. The server is stateless apart from the database and caches, so it can be scaled horizontally later.

**Availability** The failure of any single external data service must degrade a result, not abort it, and the degradation must be reported to the user in the response.

**Usability** A non-specialist must be able to run an analysis with only a village name. The interface must work with keyboard and screen readers and meet WCAG AA contrast, and target current desktop browsers (Chrome, Firefox, Safari, Edge).

**Security** All input is validated (ranges, enumerations, a 25 MB upload limit); no secret is needed to run the system; external services are called only from the server.

**Cost and portability** Only free, key-less public data; installable from pip wheels without GIS server software; deployable in a container on free hosting.

**Transparency** Every output must be traceable: the response carries data sources, methods, parameters and per-stage timings.

### 3 SYSTEM ARCHITECTURE AND HIGH-LEVEL DESIGN

The system is a three-tier web application (Figure 1). The *client* is a static single-page application (HTML, CSS, JavaScript with Leaflet and Chart.js). The *application server* is a FastAPI process that contains all analysis logic in one analysis pipeline. The *persistence tier* is an SQLite database (saved analyses and an API-response cache) plus a disk cache of map tiles. The *external API layer* consists of seven free public services, which only the server calls; this lets the server set proper User-Agent headers, cache responses and stay within fair-use limits.

A request flows as follows. The browser sends `POST /api/analyze` with a location. The server downloads terrain tiles (or reads them from the tile cache) and builds a DEM; fetches satellite imagery and OpenStreetMap features to classify land; scores and ranks sites; fetches the rainfall series (or reads it from `api_cache`); evaluates catchment, runoff and pond design for every candidate; saves the result in SQLite; and returns one JSON document with GeoJSON layers and PNG overlays, which the client draws on the map.

Figure 2 shows the stages of one analysis. The two inputs — a village location or an uploaded contour map — converge on a common in-memory DEM, after which the pipeline is identical, so a new terrain source can be added without touching the hydrology.

#### 3.1 Technology Stack

- **Language and server:** Python 3.12+ with FastAPI 0.115 and Uvicorn; Pydantic 2 for request/response models, which also generate the OpenAPI documentation at `/docs`.
- **Numerical and image processing:** NumPy, SciPy (Delaunay interpolation) and OpenCV (tile decoding, re-sampling with `cv2.remap`, contour tracing, morphology, classification, rasterisation).
- **Database:** SQLite through the Python standard library.
- **Frontend:** plain HTML/CSS/JavaScript, Leaflet 1.9 with Leaflet.draw for maps and parcels, and Chart.js 4 for charts.
- **Data services:** AWS Terrain Tiles (elevation, SRTM/Copernicus ~30 m) with Open-Meteo elevation as fallback; Esri World Imagery; OpenStreetMap Overpass and Nominatim; CHIRPS v2.0 rainfall via NASA SERVIR ClimateSERV, with Open-Meteo (ERA5) and NASA POWER as fallbacks; Open-Meteo or NASA POWER for evapotranspiration.
- **Deployment:** Docker image and a Render blueprint (`render.yaml`).

*Deviations from the suggested stack.* The brief suggests IMD for rainfall. IMD gridded rainfall is distributed as bulk files rather than through a query API, so CHIRPS was chosen after validating three datasets against IMD station

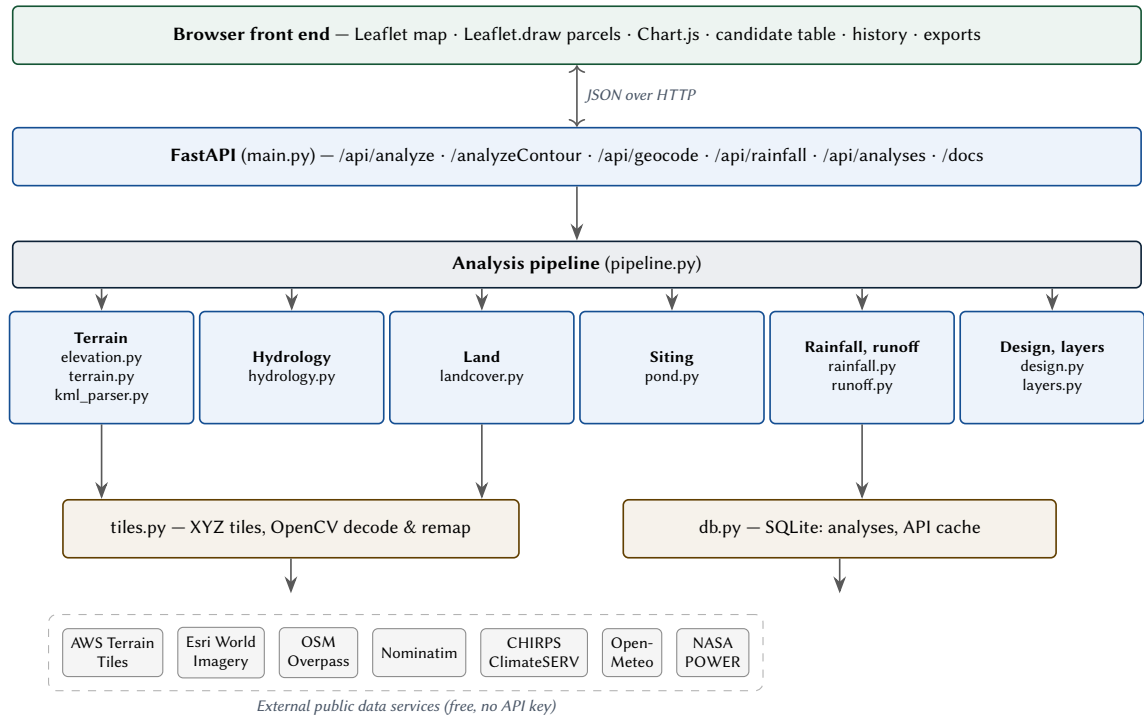


Fig. 1. High-level system architecture. Arrows show the direction of calls. Only the server talks to external services, and their responses are cached.

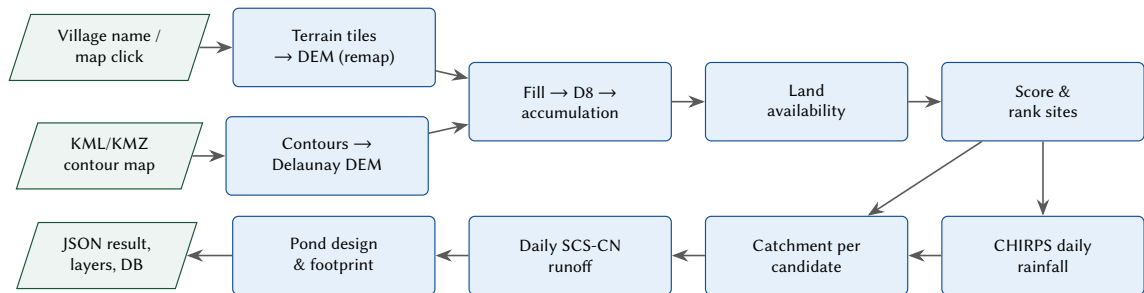


Fig. 2. Processing stages of one analysis. Rainfall is fetched once, at the top site, and shared by all candidates.

normals (Section 7): NASA POWER was off by up to +92 % in the rain shadow, CHIRPS by 8.9 % on average. We did not use GDAL or a GIS server: the required algorithms are small enough to implement directly in NumPy and OpenCV, which keeps the system installable from pip wheels on free hosting. SQLite replaces a client-server database because the data is a small document store with a single writer process.

## 4 METHODOLOGY

### 4.1 Terrain and Elevation Analysis

For a village centre and radius  $R$  (default 2 km) the server lays out an  $N \times N$  grid (default  $N = 200$ , cell size  $\Delta = 20$  m) in a local equirectangular projection. AWS Terrain Tiles encode elevation in the RGB bytes of 256×256 Web-Mercator PNG tiles as  $z = 256R + G + B/256 - 32768$  m. The tiles covering the grid are downloaded in parallel, decoded with OpenCV and stitched into a mosaic, which is then sampled at every cell centre with `cv2.remap` (bilinear). Decoding happens *before* resampling, because interpolating the packed bytes would mix the high and low bytes of neighbouring pixels and produce errors of hundreds of metres. If the tile service fails, a 40×40 lattice is queried from the Open-Meteo elevation API and interpolated bicubically.

For an uploaded KML/KMZ map, the parser detects each contour's elevation from coordinate  $z$  values, `ExtendedData`, the placemark name or the folder name (the first strategy that covers 90 % of lines with varying values wins), and the contour vertices are interpolated linearly over their Delaunay triangulation.

Contours for display are traced from the DEM with `cv2.findContours` at an automatic interval, with major and minor lines; slope and plan curvature come from finite differences. Coastal areas need one more step: the terrain tiles report the sea as a flat 0 m surface with bathymetry further out, so connected low areas containing enough sea-surface or bathymetry cells are marked as sea and excluded, while low-lying inland land (for example the polders of Kuttanad, below sea level) is kept.

### 4.2 Catchment Area Delineation

Catchments are derived from the DEM with standard flow-routing algorithms:

- (1) **Depression filling.** Priority-Flood [2] seeds a priority queue with the edge cells, repeatedly pops the lowest cell and raises each unvisited neighbour to at least its elevation plus  $10^{-4}$  m, so every cell has a downhill path to the edge ( $O(n \log n)$ ).
- (2) **Flow direction (D8).** Each cell drains to the neighbour with the steepest distance-weighted drop [3], computed as eight vectorised array shifts ( $O(n)$ ).
- (3) **Flow accumulation.** Cells are visited in decreasing filled elevation, each adding its count to its downstream neighbour ( $O(n \log n)$  for the sort).
- (4) **Catchment.** Breadth-first traversal of the reversed D8 graph from the pond cell gives the catchment mask ( $O(n)$ ). Its boundary is traced along cell edges into a GeoJSON polygon whose area equals the cell count times  $\Delta^2$ .
- (5) **Edge check.** Cells whose catchment may extend beyond the map edge are flagged; in village mode they are excluded from siting so that no catchment is truncated.

*Site selection.* Candidate cells are available-land cells on the drainage network (top 8 % of accumulation, relaxed stepwise if available land does not reach it). Each is scored

$$S = 0.75 (0.45 \hat{A} + 0.35 (1 - \hat{\theta}) + 0.20 \hat{\kappa}) + 0.25 L,$$

where  $\hat{A}$  is normalised log-accumulation,  $\hat{\theta}$  normalised slope,  $\hat{\kappa}$  normalised concavity and  $L$  the land score. Sites are then accepted best-first only if they are at least 250 m from, and neither upstream nor downstream of, every accepted site, so the inflows of the (up to five) ponds can be added without double counting.

*Available land.* Esri imagery at 5 m is classified per pixel: vegetation by the Green–Red Vegetation Index  $(G-R)/(G+R)$  with an Otsu threshold [4] clamped to  $[0, 0.10]$ ; built-up as dense clusters of bright, highly textured pixels; water as dark, blue-dominant smooth pixels; the rest as open land. OpenStreetMap buildings (+20 m buffer), roads (+4–30 m), railways, rivers, water bodies, forest, settlements and protected areas are excluded; grassland, scrub or common-land tags are preferred; parcels drawn by the administrator, if any, restrict siting to them; ground steeper than  $8^\circ$  is excluded.

*Assumptions and limitations.* D8 routes all flow to one neighbour and cannot model divergent flow; the  $\sim 30$  m source DEM cannot see bunds or drains narrower than about 30 m; and land availability is inferred, not legal ownership.

### 4.3 Rainfall Data Integration

Daily precipitation is taken from CHIRPS v2.0 [5] ( $0.05^\circ$ , satellite–gauge blend) through the ClimateSERV API, downloaded as two ten-year chunks in parallel. If CHIRPS fails the system falls back to Open-Meteo’s ERA5 reanalysis [6], then to NASA POWER. Reference evapotranspiration comes from Open-Meteo (FAO-56 Penman–Monteith) or, from NASA POWER temperatures, by Hargreaves’ equation [7]. The default record is the last 20 complete years (2006–2025), configurable from 5 to 40. If the administrator knows the local IMD normal, the daily series is scaled to it, preserving the storm pattern while removing bias.

From the daily series the system computes: mean, median, standard deviation and coefficient of variation of annual totals; the 75 %-dependable rainfall (the 25th percentile of annual totals, equalled or exceeded three years in four); the driest and wettest years; rainy days ( $\geq 2.5$  mm, the IMD definition); the heaviest day; the monsoon (June–September) share; monthly climatology; and annual and dry-season (October–May) evapotranspiration.

*Runoff and pond design.* Runoff is simulated for every day with the SCS Curve Number method [8]:  $S = 25400/CN - 254$ ,  $I_a = 0.2S$ ,  $Q = (P - I_a)^2 / (P - I_a + S)$  for  $P > I_a$ . Curve numbers come from each land-cover class for the chosen soil group, are adjusted daily to dry or wet antecedent conditions from the previous five days’ rain [9], and runoff is area-weighted over the catchment’s land cover. The pond is a rectangular excavated frustum (side slope 1.5:1, length  $1.5 \times$  width, 0.5 m freeboard) holding half of the 75 %-dependable annual runoff, so it fills about twice even in a dry year. Depth starts at 3 m (MGNREGA/Amrit Sarovar practice) and increases to at most 4.5 m if the footprint does not fit on 80 % of the connected available land; volume is computed with the prismoidal formula. The design reports earthwork, dry-season evaporation and seepage.

## 5 IMPLEMENTATION

The back end is about 3 400 lines of Python in fifteen modules; the front end about 1 200 lines of HTML, CSS and JavaScript. `main.py` handles only HTTP; `pipeline.py` orchestrates the stages; each analysis module is a function of arrays and parameters that can be tested in isolation; network and disk I/O are confined to `tiles.py`, `rainfall.py`, `landcover.py`, `geocode.py` and `db.py`.

### 5.1 Backend and API Design

The API is REST over JSON (Table 2). The full schema is generated from the Pydantic models and served at `/docs`.

*Authentication.* There is none: the system is a single-tenant planning tool that handles only public data, so every user may run and view analyses. Section 6 discusses what adding it would involve.

Table 2. HTTP endpoints

| Method       | Path                           | Purpose                                                                            |
|--------------|--------------------------------|------------------------------------------------------------------------------------|
| POST         | /api/analyze                   | Full analysis of a village location (JSON body; only lat, lon required)            |
| POST         | /analyzeContour                | Full analysis of an uploaded KML/KMZ contour map (multipart); alias /findCatchment |
| GET          | /api/geocode?q=                | Village search                                                                     |
| GET          | /api/reverse?lat=&lon=         | Place name for a map click                                                         |
| GET          | /api/rainfall?lat=&lon=&years= | Rainfall statistics only                                                           |
| GET          | /api/analyses                  | List saved analyses (summaries)                                                    |
| GET / DELETE | /api/analyses/{id}             | Retrieve or delete one analysis                                                    |
| GET          | /health                        | Liveness probe (used by the hosting platform)                                      |
| GET          | /, /docs                       | Front end; interactive API documentation                                           |

*Errors.* All errors share one JSON shape, {"status": "error", "detail": "..."}: 400 for an empty upload, 413 for an oversized one, 422 for invalid input (including a location in the open sea) and 502 when a required upstream service is down. Third-party failures are handled at three levels: (i) every source has a fallback — terrain tiles fall back to the Open-Meteo elevation API, and rainfall falls through CHIRPS, Open-Meteo and NASA POWER, with transient network errors retried with linear back-off; (ii) optional evidence (imagery, OpenStreetMap) may fail without aborting the analysis, and the loss is added to the response's warnings; (iii) if all rainfall sources fail, the rational method is applied to a fallback rainfall value, again with a warning.

## 5.2 Frontend and Visualization

The front end (Figure 3) is a single page with *Village*, *Contour file* and *History* tabs over a Leaflet map. The Village tab offers search, a preview of the analysis square, parcel drawing and GeoJSON upload with Leaflet.draw, an optional manual pond pin and advanced parameters (soil group, rainfall dataset and normal, storage fraction, depth and slope limits). While an analysis runs, a staged progress overlay covers the map. Results appear as a headline storage card and four key figures, a ranked candidate table mirrored by numbered map markers (selecting a candidate redraws everything for it), and *Pond*, *Rainfall*, *Runoff* and *Site & land* sub-tabs with a pond cross-section and Chart.js charts. Map layers — contours, drainage, catchment, available land, land-cover and relief rasters, pond footprints — can be toggled. Results export as JSON or GeoJSON and print through a dedicated stylesheet. Accessibility work includes ARIA tab semantics, keyboard operation, visible focus, a skip link and AA text contrast.

## 5.3 Database and Storage

SQLite holds two tables. analyses stores every completed analysis twice: a compact summary (summary\_json) for the history list and the full response (result\_json) for exact re-display, together with the village, mode, the top site's coordinates and a UTC timestamp; an index on created\_at serves the newest-first history query, and single analyses are fetched by primary key. api\_cache is a key-value table (key, created\_at, JSON payload) that memoises external responses with a time-to-live: rainfall series for 30 days (keyed by source, location rounded to 0.01° and years), OpenStreetMap features for 7 days (keyed by bounding box) and geocoding for 7 days. Map tiles are cached as files keyed by source, zoom and tile coordinates. Spatial data (catchments, footprints, land polygons) is stored as GeoJSON inside



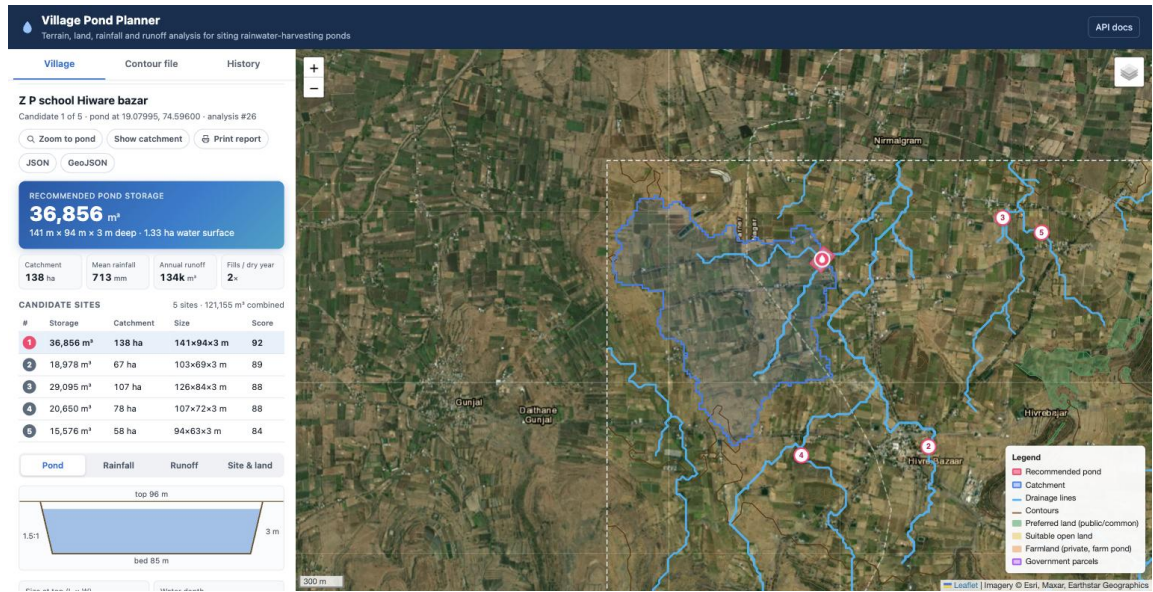


Fig. 3. Application interface showing the results overlay for Hiware Bazar.

the result document rather than in spatial tables, because it is only ever read back as a whole. All access goes through `db.py`, which serialises writes with a lock.

## 6 CSD THEMES AND TOPICS APPLIED IN THE PROJECT

Table 3 maps the Computer System Design themes we applied to concrete parts of the system. Themes we did not implement are listed after the table with the reason.

*Themes not implemented.* **Load balancing:** one Unicorn process serves the expected load (a few planners); the bottleneck is external APIs, not CPU, and SQLite favours a single writer process. Scaling out would mean running several workers behind a reverse proxy such as Nginx and moving the database and caches to PostgreSQL and Redis. **Authentication and authorisation:** all data is public and the tool is single-tenant; a multi-panchayat deployment would need login (e.g. OAuth) and per-village ownership of saved analyses. **Version control / CI-CD:** the code is kept in a Git repository (Appendix A); there is no CI pipeline yet — running the unit tests in GitHub Actions on every push would be the first step.

The three most significant decisions were the following. **(1) Caching plus resilience around external APIs.** Every slow or fragile dependency sits behind a cache and a fallback. Load testing showed why both are needed: with four concurrent analyses, uncached OpenStreetMap queries exceeded Overpass's per-IP limit, each request waited about 44s and then lost all its OpenStreetMap exclusions. We added a 7-day cache, serialised the calls and added a circuit-breaker cool-down, after which a failing Overpass costs one timeout instead of one per request. **(2) A modular monolith instead of microservices.** The stages pass large arrays between them, the team is one person and the hosting is a single free instance; a monolith with strict module boundaries gives the testability of services without their operational cost. **(3) Choosing data by measurement.** The rainfall source was initially Open-Meteo with NASA



Table 3. Mapping of core CSD themes/topics to their use in this project

| CSD Theme/Topic                        | Where used in the project                                                                                                                                                                                                | Justification / design rationale                                                                                                                                                                                                                                                                         |
|----------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| API Design (REST)                      | /api/analyze, /analyzeContour, /api/rainfall, /api/analyses/{id}; Pydantic schemas; OpenAPI at /docs                                                                                                                     | Resources map naturally to REST (an analysis, a rainfall summary, a saved record). One request returns one self-describing document (results, layers, warnings, methods, timings), so the client needs no follow-up calls. The schema is the contract and is validated on every request.                 |
| Caching                                | Tile files on disk; api_cache table (rainfall 30 d, OSM 7 d, geocoding 7 d); saved analyses                                                                                                                              | External APIs dominate latency and are rate-limited. A cached analysis takes 1.3 s instead of 44 s. Caching OpenStreetMap was added after load testing showed concurrent requests exhausting the Overpass quota (Section 7.1).                                                                           |
| Database Indexing / Query Optimization | Primary keys on analyses.id and api_cache.key; index on analyses(created_at); summary stored separately from the full result                                                                                             | The two hot queries are “newest $n$ analyses” and “cache lookup by key”; both are index scans. The history list reads only the small summary column, not multi-megabyte results. No spatial index is needed because no query searches by location.                                                       |
| Concurrency / Asynchronous Processing  | Thread pools: 8 parallel tile downloads; two CHIRPS decades in parallel; rainfall and ET <sub>0</sub> in parallel; FastAPI runs each sync request in its worker thread pool; database lock; a lock around Overpass calls | The work is I/O-bound on slow APIs, so overlapping downloads cuts first-run time; NumPy and OpenCV release the GIL, so concurrent analyses also run in parallel (8 concurrent cached analyses in 1.3 s each). Serialising Overpass calls respects its per-IP limit.                                      |
| Microservices vs. Monolith             | Modular monolith: one FastAPI process, layered modules                                                                                                                                                                   | One developer, one deployable unit, and stages that exchange large in-memory arrays (40 000-cell grids) — splitting them into services would add serialisation and network hops for no gain. Module boundaries (pure functions of arrays) keep a later split possible, e.g. a separate rainfall service. |
| Design Patterns                        | Pipeline (pipeline.run); fallback chain of data providers (rainfall, elevation); circuit breaker (Overpass cool-down); data-access module (db.py); DTOs (Pydantic models)                                                | The pipeline makes each stage replaceable; the fallback chain implements graceful degradation uniformly; the circuit breaker stops one failing service from queueing every request behind a 36 s timeout; isolating SQL in one module would let SQLite be swapped for PostgreSQL.                        |
| Containerization / Deployment          | Dockerfile (python:3.12-slim), render.yaml blueprint with health check                                                                                                                                                   | Reproducible environment for grading and demo; one command to run; the health endpoint lets the platform restart a failed instance.                                                                                                                                                                      |
| Error Handling and Resilience          | Fallbacks for every source; retries with back-off; warnings in the response; uniform error JSON; input validation; 25 MB upload cap                                                                                      | Public APIs are rate-limited and sometimes down; the system must degrade, not fail, and must tell the user exactly what was degraded.                                                                                                                                                                    |
| Algorithms and Complexity              | Priority-Flood $O(n \log n)$ ; vectorised D8 $O(n)$ ; sorted accumulation $O(n \log n)$ ; BFS catchment $O(n)$ ; best-first independent ranking                                                                          | Terrain analysis on 40 000 cells takes about 60 ms, so algorithmic cost is negligible next to I/O. Priority-Flood was chosen over iterative pit filling, which is $O(n^2)$ in the worst case.                                                                                                            |
| Testing Strategy                       | 35 tests (30 unit, 5 API); validation scripts against independent data; end-to-end and load tests through the HTTP API                                                                                                   | Unit tests fix formulas and the hydrology core against analytic answers; validation measures accuracy against IMD, survey and hand-labelled data; system tests caught a bug (ponds placed in the sea) and a concurrency failure, both fixed.                                                             |

POWER as fallback; validation against IMD normals showed errors up to 92 %, so CHIRPS became the default and the others fallbacks — an architectural decision driven by data quality rather than convenience.

## 7 RESULTS AND EVALUATION

*Case study: Hiware Bazar.* Hiware Bazar (Ahilyanagar district, Maharashtra) is a model watershed village on the semi-arid Deccan plateau. We analysed a 4 km×4 km area centred at 19.068°N, 74.601°E with soil group C and default parameters. The figures below are from a run on 27 September 2026; because imagery, OpenStreetMap and CHIRPS are live sources, later runs can differ slightly (a rerun on 2 October, after one road was removed from OpenStreetMap, reordered the two nearly equal leading sites).

The DEM spans 692–875 m. After exclusions, 76 % of the area (1 219 ha) is suitable for excavation. CHIRPS gives a mean annual rainfall of 713 mm for 2006–2025, a 75 %-dependable rainfall of 599 mm and a coefficient of variation of 23 % (from 373 mm in 2018 to 1 063 mm in 2020); 79 % falls in June–September (Figure 4). The ranking algorithm found five independent sites, all on open land with slopes of 0.1–1.6° (Table 4).

Table 4. Ranked candidate ponds for Hiware Bazar (soil group C, CHIRPS 2006–2025)

| Rank         | Score | Location (°N, °E)  | Catchment (ha) | Mean runoff (m <sup>3</sup> /yr) | Dep. runoff (m <sup>3</sup> /yr) | Storage (m <sup>3</sup> ) | Top size (m) |
|--------------|-------|--------------------|----------------|----------------------------------|----------------------------------|---------------------------|--------------|
| 1            | 0.92  | 19.07995, 74.59600 | 137.6          | 134 368                          | 73 712                           | 36 856                    | 141×94       |
| 2            | 0.89  | 19.06862, 74.60285 | 67.3           | 68 007                           | 37 955                           | 18 978                    | 103×69       |
| 3            | 0.88  | 19.08247, 74.60760 | 106.5          | 105 393                          | 58 190                           | 29 095                    | 126×84       |
| 4            | 0.88  | 19.06808, 74.59466 | 78.5           | 75 657                           | 41 301                           | 20 650                    | 107×72       |
| 5            | 0.84  | 19.08157, 74.61008 | 57.7           | 56 694                           | 31 153                           | 15 576                    | 94×63        |
| <b>Total</b> |       |                    | <b>447.6</b>   | <b>440 119</b>                   |                                  | <b>121 155</b>            |              |

The recommended pond (rank 1) lies on open land at 709 m with a 0.4° slope, at the outlet of a 137.6 ha catchment (perimeter 7.72 km, relief 44.8 m). The catchment is 77 % open or fallow land, 22 % crops and 1 % built-up (composite CN 85.8), giving a mean runoff of 98 mm/yr — an effective runoff coefficient of 0.14 — or 134 368 m<sup>3</sup>/yr, and 73 712 m<sup>3</sup> in a dependable year. The design is a 141.3 m×94.2 m pond, 3.0 m deep plus 0.5 m freeboard, storing 36 856 m<sup>3</sup> with 43 604 m<sup>3</sup> of excavation; it fills 2.0 times in a dependable year, and dry-season evaporation removes about 44 % of its storage, which is why the design prefers depth to area once land is limited.

*Comparison with reference data.* Each component was checked against independent references (Table 5). Against IMD 1991–2020 normals at six observatories, CHIRPS has a mean absolute error of 8.9 % (Ahmednagar, the nearest station to Hiware Bazar, +8.2 %), while NASA POWER has 26.1 % and overestimates Pune by 92 %. Terrain-tile elevations at 44 surveyed summits have a median absolute error of 20 m, a worst case since a 30 m DEM smooths sharp peaks. The land classifier was 92 % accurate ( $\kappa = 0.85$ ) on 87 unmixed, blind hand-labelled cells in four landscapes; the sample contained no water cells, so water detection is not validated. Runoff could not be compared with measured flow because no gauged village-scale catchments exist in open data; the effective coefficient of 0.14 is plausible for a semi-arid catchment but unverified. A sensitivity analysis shows runoff has an elasticity of about three with respect to rainfall ( $\pm 10$  % rain gives  $-30/+31$  % runoff) and varies by a factor of 4.7 between soil groups A and D, so rainfall and soil group dominate the uncertainty.

### 7.1 Performance

All timings were measured through the HTTP API on an Apple M4 laptop (10 cores), 2–4 October 2026 (Table 6).

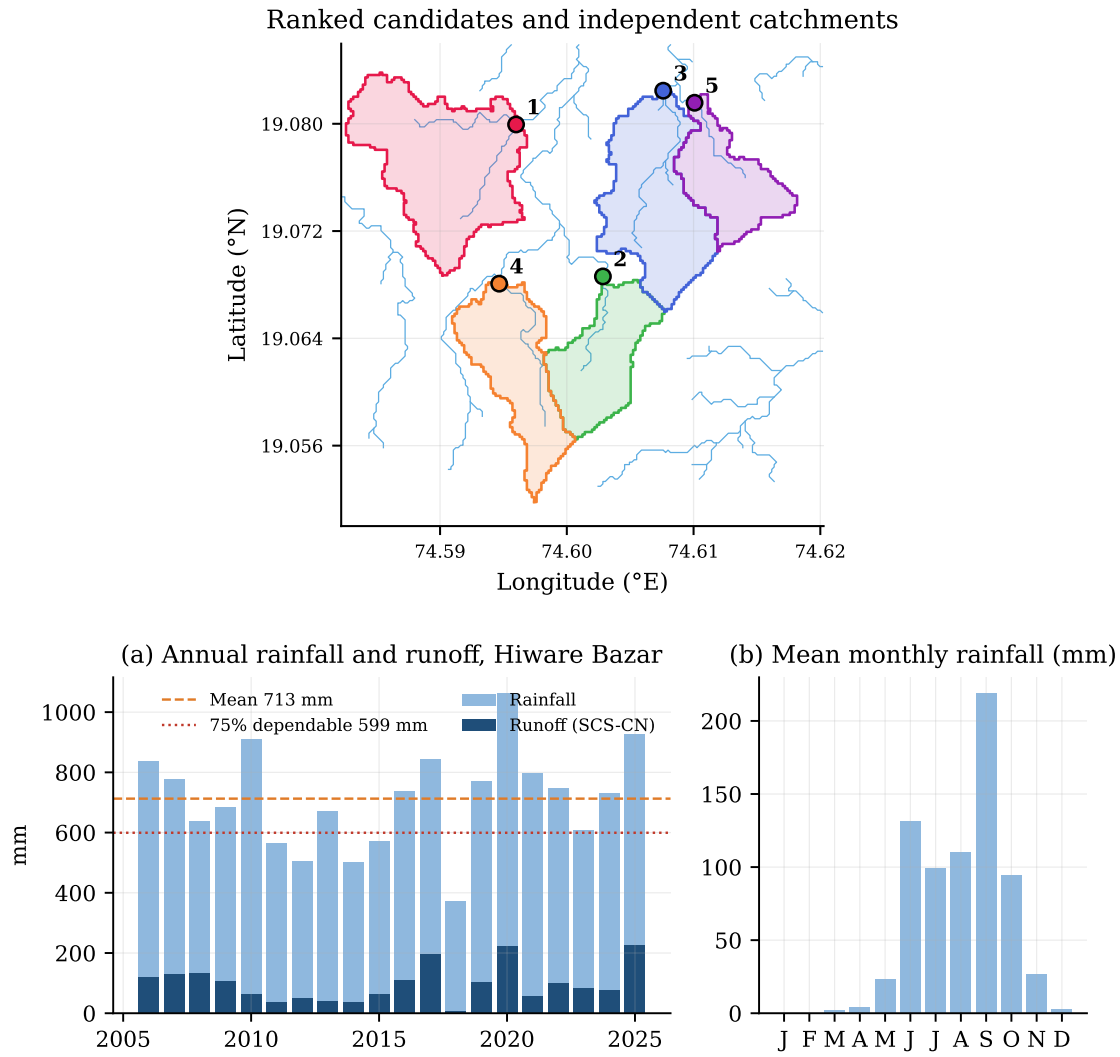


Fig. 4. Example results for Hiware Bazar: (top) the five ranked sites and their independent catchments; (bottom) annual rainfall and simulated runoff depth 2006–2025, and the monthly rainfall climatology.

A first analysis is dominated by the rainfall download (about 35 s for two parallel ten-year CHIRPS requests); a repeat is dominated by fetching and classifying imagery. The computation itself is fast: terrain analysis of 40 000 cells takes 60 ms because D8 is vectorised and accumulation is a single sorted pass.

*Behaviour under load.* Load testing exposed a real problem. With the original code, four concurrent cached analyses each took 45 s instead of 1.3 s and all of them returned without OpenStreetMap data: OpenStreetMap queries were not cached, the public Overpass server admits only about two queries per IP at a time, and the others timed out. We added a 7-day cache, serialised Overpass calls and a 120 s cool-down after a failure. During our tests Overpass then refused

Table 5. Validation summary

| Component              | Reference                                                      | Result                                                      |
|------------------------|----------------------------------------------------------------|-------------------------------------------------------------|
| Formulas and hydrology | Hand calculations, analytic catchment, FAO-56 / TR-55 examples | 35/35 tests pass; synthetic catchment matched cell for cell |
| Rainfall (CHIRPS)      | IMD 1991–2020 normals, 6 stations                              | Mean absolute error 8.9 % (NASA POWER 26.1 %)               |
| Elevation              | 44 surveyed summits, 3 regions                                 | Median error 20 m, bias –27 m                               |
| Land-cover classifier  | 100 blind-labelled 20 m cells, 4 landscapes                    | 92 % accuracy, $\kappa = 0.85$                              |

Table 6. Measured response times

| Operation                                                   | Time                      |
|-------------------------------------------------------------|---------------------------|
| Village analysis, first run (20 years of CHIRPS downloaded) | 44.2 s                    |
| Village analysis, cached rainfall and tiles                 | 1.3 s                     |
| of which terrain, fill, D8, accumulation (40 000 cells)     | 0.06 s                    |
| of which land assessment (imagery classification + OSM)     | 1.1 s                     |
| of which ranking, 5 catchments, runoff, design, layers      | 0.09 s                    |
| Contour-map analysis (sample_contours.kml), first run       | 64.6 s                    |
| GET /api/rainfall, cached                                   | 8 ms                      |
| GET /api/geocode, cached                                    | 1 ms                      |
| GET /health, 50 concurrent requests                         | median 2.4 ms, p95 3.3 ms |
| 8 concurrent cached village analyses                        | median 1.32 s, max 1.35 s |

connections from our network altogether — a temporary block, most likely triggered by the load test itself — which let us test the failure path: a batch of four concurrent requests shared a single 36 s timeout (previously up to 293 s for the last request), and the following eight concurrent requests completed in 1.3 s each, with the missing OpenStreetMap data reported in their warnings. Because NumPy and OpenCV release Python’s global interpreter lock, eight concurrent analyses took no longer than one: the CPU is not the bottleneck; external APIs are. The successful-cache path is covered by unit tests but could not be load-tested live while Overpass blocked our network.

## 8 DISCUSSION AND LIMITATIONS

*What worked well.* Building everything on one in-memory DEM made the two input modes share all downstream code. Validating data sources early changed the design for the better (the switch to CHIRPS). Returning methods, timings and warnings with every result made failures visible instead of silent, and end-to-end testing found two bugs that unit tests missed.

*What did not.* The original RGB water rule (blue brighter than red and green) does not recognise the turbid, green-tinted water typical of Indian coasts and reservoirs; before we fixed it, the system recommended ponds offshore for a Konkan coastal village and even on the Pacific seabed. Sea is now excluded using the elevation data, but inland water still relies on OpenStreetMap. The dependence on live public services makes results drift as those services change, and some services rate-limit aggressively.

*Data quality and assumptions.* The ~30 m DEM cannot resolve field bunds, drains or road embankments, which can redirect flow on the ground. CHIRPS underestimates deserts (–28 % at Jaisalmer). Soil group is a user input and is the

most influential parameter after rainfall. The SCS-CN method with TR-55 curve numbers was developed for US catchments; its use in India is standard in watershed manuals but not locally calibrated. Land ownership is inferred from tags and imagery, never certain, and groundwater depth and rock are ignored when choosing depth. All recommendations therefore need field verification.

## 9 AI TOOL USAGE DECLARATION

**[TODO: List every AI tool you used while developing the project (e.g. ChatGPT, Claude, Gemini, GitHub Copilot) and what you used each for. The paragraph below covers only the final review session.]**

Claude Code (Anthropic, model Claude Opus 5.5) was used in the final stage of the project for: reviewing the code and running the application end to end; finding and fixing two defects (pond sites placed in the sea, and lost OpenStreetMap data under concurrent load), including the unit tests for both fixes; measuring the performance figures in Section 7.1; updating the README, validation notes and the main report; and reformatting this report into the ACM template from the existing technical report. **[TODO: Confirm honestly: “All AI-assisted code and text was reviewed, understood and, where needed, modified by me before submission.”]**

## REFERENCES

- [1] Ministry of Rural Development, Government of India. 2022. *Mission Amrit Sarovar: Guidelines*.
- [2] R. Barnes, C. Lehman, and D. Mulla. 2014. Priority-flood: An optimal depression-filling and watershed-labeling algorithm for digital elevation models. *Computers & Geosciences* 62, 117–127.
- [3] J. F. O’Callaghan and D. M. Mark. 1984. The extraction of drainage networks from digital elevation data. *Computer Vision, Graphics, and Image Processing* 28, 3, 323–344.
- [4] N. Otsu. 1979. A threshold selection method from gray-level histograms. *IEEE Transactions on Systems, Man, and Cybernetics* 9, 1, 62–66.
- [5] C. Funk et al. 2015. The climate hazards infrared precipitation with stations — a new environmental record for monitoring extremes. *Scientific Data* 2, 150066.
- [6] H. Hersbach et al. 2020. The ERA5 global reanalysis. *Quarterly Journal of the Royal Meteorological Society* 146, 730, 1999–2049.
- [7] R. G. Allen, L. S. Pereira, D. Raes, and M. Smith. 1998. *Crop Evapotranspiration: Guidelines for Computing Crop Water Requirements*. FAO Irrigation and Drainage Paper 56, Rome.
- [8] USDA Soil Conservation Service. 1986. *Urban Hydrology for Small Watersheds*. Technical Release 55, 2nd ed.
- [9] R. H. Hawkins, A. T. Hjelmfelt, and A. W. Zevenbergen. 1985. Runoff probability, storm depth, and curve numbers. *Journal of Irrigation and Drainage Engineering* 111, 4, 330–340.

## A SOURCE CODE AND REPOSITORY

Repository: <https://github.com/amandeep01/village-pond-planner> **[TODO: check this link is public and up to date].**

**app/** Back end: `main.py` (routes), `schemas.py` (API models), `pipeline.py` (orchestration), `elevation.py`, `tiles.py`, `terrain.py`, `kml_parser.py` (terrain), `hydrology.py`, `landcover.py`, `pond.py`, `rainfall.py`, `runoff.py`, `design.py`, `layers.py` (analysis), `geocode.py`, `db.py` (I/O).

**app/static/** Front end: `index.html`, `app.css`, `app.js`.

**tests/** Unit tests (pytest).

**validation/** Scripts and results for the rainfall, elevation and land-cover validation.

**report/** Reports, figures and the script that regenerates them.

**Root** `README.md` (usage, API, methods), `VALIDATION.md`, `requirements.txt`, `requirements-dev.txt`, `Dockerfile`, `render.yaml` and `sample_contours.kml`.

To run: `pip install -r requirements.txt`, then `uvicorn app.main:app --port 8000` and open <http://127.0.0.1:8000>.